# CSS 262: Linux Administration

## SSH Hardening & Remote Access

Lecture 8: Securing Remote Access

# 📋 Today's Agenda

## Part 1: SSH Fundamentals

- What is SSH & why it matters
- Client/server architecture
- Key exchange & encryption
- Basic usage & file transfer

## Part 2: Key-Based Authentication

- Why keys beat passwords
- Key types & generation
- Deploying & managing keys
- ssh-agent workflow

## Part 3: Hardening sshd_config

- Critical security settings
- Restricting access
- Testing safely

## Part 4: Advanced SSH

- SSH config aliases
- Tunneling & port forwarding
- Fail2Ban & audit tools

# 🔄 Quick Recap: Week 7

## Package Management

- **apt** (Debian/Ubuntu) & **dnf** (Fedora/RHEL)
- Repository management ( `/etc/apt/sources.list` , `.repo` files)
- Package queries: `apt search` , `dnf info`
- Dependency resolution & version pinning
- Building from source: `./configure && make && make install`

## Key Takeaways

- Always prefer repo packages over manual installs
- Keep systems updated: `apt upgrade` / `dnf upgrade`
- GPG keys verify package integrity
- `checkinstall` tracks manual builds
- **Today**: We secure the door to your server – SSH

🔐 **Security Block Begins:** From this week forward, we focus on hardening Linux systems against real-world attacks.

# Part 1

🌐 SSH Fundamentals

# 🌐 What is SSH?

## Secure Shell Protocol

- Cryptographic network protocol (RFC 4253)
- Default port: **22/TCP**
- Replaces insecure protocols: Telnet, rlogin, rsh
- Encrypted communication channel
- Authentication + confidentiality + integrity

## What SSH Provides

- **Remote shell access** – execute commands
- **File transfer** – SCP, SFTP
- **Port forwarding** – secure tunnels
- **X11 forwarding** – remote GUI apps
- **Key-based auth** – passwordless login

⚠️ **Why this matters:** SSH is the #1 target for brute-force attacks on internet-facing Linux servers. Shodan indexes millions of exposed SSH services.

# 🏗️ SSH Architecture

## Client / Server Model

- **Server**: `sshd` daemon listens on port 22
- **Client**: `ssh` command initiates connection
- **OpenSSH**: De-facto standard implementation
- Config: `/etc/ssh/sshd_config` (server), `~/.ssh/config` (client)

## Connection Flow

1. TCP handshake on port 22
2. Protocol version exchange
3. **Key exchange** (Diffie-Hellman)
4. Server authentication (host key)
5. **User authentication** (password/key)
6. Encrypted session established

🔑 **Host keys** live in `/etc/ssh/ssh_host_*` – they prove the server's identity. First-connection prompt = TOFU (Trust On First Use).

# 🔐 How SSH Encryption Works

## Three Layers of Security

| Layer | Purpose | Algorithms |
|---|---|---|
| **Transport** | Encryption + integrity | AES-256-GCM, ChaCha20 |
| **Authentication** | Verify user identity | Public key, password, GSSAPI |
| **Connection** | Multiplex channels | Sessions, forwarded ports |

## Key Exchange (Simplified)

1. Client & server agree on crypto algorithms
2. **Diffie-Hellman** generates shared session key
3. Neither side transmits the key – derived independently
4. Session key encrypts all subsequent traffic

✅ Even if an attacker captures all traffic, they cannot decrypt it without the ephemeral session key.

# 💻 Basic SSH Usage

## Connecting to a Remote Host

```
ssh user@192.168.1.100            # Basic connection
ssh -p 2222 user@host.example.com # Custom port
ssh -v user@host                  # Verbose (debugging)
ssh user@host 'uptime && df -h'   # Run command remotely
```

## First Connection – Host Key Verification

```
The authenticity of host '192.168.1.100' can't be established.
ED25519 key fingerprint is SHA256:xR4kL9m2pQw ...
Are you sure you want to continue connecting (yes/no)?
```

✅ **Do:** Verify fingerprint out-of-band before accepting

❌ **Don't:** Blindly type "yes" – MITM attacks exploit this

# 📂 SCP & SFTP – Secure File Transfer

## SCP (Secure Copy)

```
# Upload file
scp file.txt user@host:/tmp/

# Download file
scp user@host:/var/log/syslog ./

# Recursive directory copy
scp -r ./project user@host:~/

# Custom port
scp -P 2222 file.txt user@host:/tmp/
```

## SFTP (SSH File Transfer Protocol)

```
sftp user@host
sftp> ls
sftp> cd /var/log
sftp> get syslog
sftp> put localfile.txt
sftp> exit
```

💡 SFTP is interactive and supports resume. Prefer SFTP over SCP for large transfers.

# Part 2

🔑 Key-Based Authentication

# 🔑 Why Keys > Passwords

## Password Problems

- Brute-force attacks (hydra, medusa)
- Dictionary attacks
- Credential stuffing from leaks
- Keyloggers & shoulder surfing
- Users reuse passwords across systems
- Difficult to audit & rotate

## Key Advantages

- **2048+ bit** keys – infeasible to brute-force
- No secret transmitted over the network
- Supports passphrase for 2nd factor
- Easy automation (CI/CD, scripts)
- Per-device keys enable fine-grained revocation
- Can restrict commands per key

🚨 **Real-world stat:** SSH brute-force attempts average 10,000+/day on exposed servers. Password auth = open invitation.

# 🔐 Key Types Comparison

| Type | Key Size | Speed | Security | Recommendation |
|---|---|---|---|---|
| **RSA** | 2048-4096 bit | Slower | Good (≥3072) | Legacy compatibility |
| **DSA** | 1024 bit | Fast | ❌ Deprecated | **Never use** |
| **ECDSA** | 256/384/521 bit | Fast | Good | Avoid (NIST curve concerns) |
| **Ed25519** | 256 bit | Fastest | Excellent | ✅ **Recommended** |

✅ **Use Ed25519:** Shorter keys, faster operations, no known weaknesses, resistant to side-channel attacks.

⚠️ **Exception:** Use RSA-4096 only if connecting to legacy systems that don't support Ed25519.

# 🛠️ Generating SSH Keys

## Generate an Ed25519 Key Pair

```
ssh-keygen -t ed25519 -C "student@css262-lab"
```

```
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/user/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/user/.ssh/id_ed25519
Your public key has been saved in /home/user/.ssh/id_ed25519.pub
```

## What Gets Created

| File | Contents | Share? |
|---|---|---|
| `~/.ssh/id_ed25519` | Private key | ❌ **NEVER** |
| `~/.ssh/id_ed25519.pub` | Public key | ✅ Yes – deploy to servers |

🚨 **Always set a passphrase!** An unprotected private key = anyone with file access can impersonate you.

# 📤 Deploying Public Keys

## Method 1: ssh-copy-id (Easy)

```
ssh-copy-id user@192.168.1.100
```

Appends your public key to the remote `~/.ssh/authorized_keys`.

## Method 2: Manual

```
cat ~/.ssh/id_ed25519.pub | \
  ssh user@host \
  'mkdir -p ~/.ssh && \
   cat >> ~/.ssh/authorized_keys'
```

## Critical Permissions

```
# On remote server:
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
chmod 600 ~/.ssh/id_ed25519
chmod 644 ~/.ssh/id_ed25519.pub
```

🚨 **Wrong permissions = SSH silently refuses key auth!** This is the #1 debugging issue for students.

💡 `StrictModes yes` in sshd_config enforces permission checks.

# 🔒 ssh-agent – Key Management

## The Problem

Typing your passphrase every time is tedious – but removing it is insecure.

## The Solution: ssh-agent

```
eval "$(ssh-agent -s)"      # Start the agent
ssh-add ~/.ssh/id_ed25519   # Add key (enter passphrase once)
ssh-add -l                  # List loaded keys
ssh user@host               # No passphrase prompt!
```

## Agent Forwarding (Use with Caution)

```
ssh -A user@jumpbox         # Forward agent to remote host
```

✅ Agent keeps keys in memory – passphrase entered once per session

⚠️ Agent forwarding is risky – compromised remote host can use your keys. Prefer `ProxyJump` instead.

# Part 3

🛡️ Hardening sshd_config

# 📄 The sshd_config File

## Location: `/etc/ssh/sshd_config`

```
sudo nano /etc/ssh/sshd_config    # Edit config
sudo sshd -t                       # Test syntax before restart
sudo systemctl restart sshd        # Apply changes
```

## Config Format

```
# Comments start with #
Keyword Value
PermitRootLogin no
Port 2222
```

🚨 **Golden Rule:** Always keep a second SSH session open while testing config changes. Lock yourself out = bad day.

# 🚫 Disable Root Login & Passwords

## PermitRootLogin

```
# /etc/ssh/sshd_config
PermitRootLogin no
```

- `no` – root cannot SSH in at all ✅
- `prohibit-password` – keys only for root
- `yes` – full access ❌ dangerous
- Use `sudo` after logging in as normal user

## PasswordAuthentication

```
# /etc/ssh/sshd_config
PasswordAuthentication no
PubkeyAuthentication yes
```

- Disables password login entirely
- Only key-based auth allowed
- Eliminates brute-force risk
- **Prerequisite:** deploy keys first!

⚠️ **Order of operations:** 1) Generate keys → 2) Deploy to server → 3) Test key login → 4) THEN disable passwords. Never reverse this!

# 🔒 Port, Access & Rate Limiting

```
# /etc/ssh/sshd_config

Port 2222                    # Change from default 22
AllowUsers alice bob         # Only these users can SSH
AllowGroups sshusers         # Or restrict by group
MaxAuthTries 3               # Lock out after 3 failed attempts
LoginGraceTime 30            # 30 seconds to authenticate
ClientAliveInterval 300      # Disconnect idle sessions (5 min)
ClientAliveCountMax 2        # After 2 missed keepalives
```

- **Port change**: Security through obscurity – reduces noise, not a real defense
- **AllowUsers/Groups**: Explicit allowlist > implicit deny
- **MaxAuthTries**: Slows brute-force attempts
- **LoginGraceTime**: Prevents hanging connections
- **ClientAlive**: Kills zombie sessions
- **DenyUsers/DenyGroups** also available

# 🛡️ Additional Hardening Options

```
# /etc/ssh/sshd_config — continued

X11Forwarding no              # Disable unless needed
AllowTcpForwarding no         # Prevent tunnel abuse
PermitEmptyPasswords no       # Never allow empty passwords
Banner /etc/ssh/banner.txt    # Legal warning banner
MaxSessions 3                 # Limit multiplexed sessions
```

## SSH Protocol Version

```
Protocol 2                    # Only allow SSHv2 (v1 is broken)
```

💡 **Banner tip:** Display a legal notice — "Unauthorized access is prohibited." Required for legal prosecution in many jurisdictions.

🚨 **SSHv1:** Vulnerable to session hijacking. Any system still using v1 is critically insecure.

# 🔄 Apply & Test Changes Safely

## Step-by-Step Process

```
# 1. Backup current config
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak

# 2. Edit config
sudo nano /etc/ssh/sshd_config

# 3. Validate syntax
sudo sshd -t
# If no output = config is valid

# 4. Restart the service
sudo systemctl restart sshd

# 5. Test in a NEW terminal (keep current session open!)
ssh -p 2222 user@host
```

🚨 **Never close your existing session until you verify the new config works!** If you lock yourself out, you'll need physical/console access to recover.

# 📊 sshd_config Quick Reference

| Setting | Value | Why |
|---|---|---|
| `PermitRootLogin` | `no` | Force sudo, audit trail |
| `PasswordAuthentication` | `no` | Eliminate brute-force |
| `PubkeyAuthentication` | `yes` | Strong crypto auth |
| `Port` | Non-default | Reduce scan noise |
| `AllowUsers` | Explicit list | Minimise attack surface |
| `MaxAuthTries` | `3` | Rate-limit failures |
| `LoginGraceTime` | `30` | Drop slow connections |
| `X11Forwarding` | `no` | Reduce attack surface |
| `PermitEmptyPasswords` | `no` | Block misconfigured accounts |

# Part 4

🚀 Advanced SSH

# ⚙️ SSH Client Config File

## `~/.ssh/config` – Aliases for Connections

```
Host lab-server
    HostName 192.168.1.100
    User student
    Port 2222
    IdentityFile ~/.ssh/id_ed25519

Host jump
    HostName bastion.example.com
    User admin

Host internal
    HostName 10.0.1.50
    User admin
    ProxyJump jump
```

## Usage

```
ssh lab-server    # Expands to full ssh command with all options
ssh internal      # Automatically jumps through bastion
```

# 🚇 SSH Tunneling / Port Forwarding

## Local Forward ( `-L` )

Access remote service locally:

```
ssh -L 8080:localhost:80 user@server
# localhost:8080 → server:80
```

```
[You] :8080 ──SSH──▶ [Server] :80
```

**Use case:** Access a remote web app through an encrypted tunnel.

## Remote Forward ( `-R` )

Expose local service to remote:

```
ssh -R 9090:localhost:3000 user@server
# server:9090 → your-machine:3000
```

```
[Server] :9090 ──SSH──▶ [You] :3000
```

**Use case:** Let remote server access your local dev environment.

💡 **Dynamic SOCKS proxy:** `ssh -D 1080 user@server` – routes all traffic through the SSH tunnel (poor man's VPN).

# Fail2Ban – Brute-Force Protection

## What It Does

- Monitors log files for failed login attempts
- Bans offending IPs via firewall rules
- Configurable thresholds and ban duration
- Essential for any internet-facing SSH server

## Install & Enable

```
sudo apt install fail2ban
sudo systemctl enable --now fail2ban
```

## SSH Jail Config

```
# /etc/fail2ban/jail.local
[sshd]
enabled  = true
port     = 2222
filter   = sshd
logpath  = /var/log/auth.log
maxretry = 3
bantime  = 3600
findtime = 600
```

## Useful Commands

```
sudo fail2ban-client status sshd
sudo fail2ban-client unban <IP>
```

# 🔍 SSH Audit & Two-Factor Auth

## SSH Audit Tools

```
# ssh-audit — check server config
pip install ssh-audit
ssh-audit 192.168.1.100
```

- Reports weak algorithms & ciphers
- Checks key exchange methods
- Flags vulnerable configurations
- Run against your own servers regularly

## Two-Factor Auth (2FA)

```
sudo apt install libpam-google-authenticator
google-authenticator
```

- Adds TOTP (time-based one-time password)
- Works with Google Authenticator, Authy, etc.
- Requires PAM configuration changes
- SSH key + OTP = very strong auth

✅ **Defense in depth:** Key-based auth + Fail2Ban + 2FA + non-default port = multiple layers an attacker must bypass.

# ✅ SSH Security Checklist

## Authentication

- ☐ Use Ed25519 keys
- ☐ Set passphrases on private keys
- ☐ `PasswordAuthentication no`
- ☐ `PermitRootLogin no`
- ☐ `PubkeyAuthentication yes`
- ☐ Deploy keys with correct permissions

## Server Hardening

- ☐ Change default port
- ☐ `AllowUsers` / `AllowGroups`
- ☐ `MaxAuthTries 3`
- ☐ `LoginGraceTime 30`
- ☐ `X11Forwarding no`
- ☐ Install & configure Fail2Ban
- ☐ Run `ssh-audit` and fix findings
- ☐ Set a legal banner

🚨 **Remember:** Security is a layered approach. SSH hardening is one piece – combine with firewalls (next week), updates, and monitoring.

# 📊 Summary: SSH Hardening & Remote Access

## Key Concepts Covered

1. **SSH Protocol**: Encrypted remote access on port 22
2. **Architecture**: Client/server, key exchange, host keys
3. **Key Auth**: Ed25519, ssh-keygen, ssh-copy-id
4. **Permissions**: 700 (.ssh), 600 (keys)
5. **sshd_config**: Disable root, disable passwords
6. **Fail2Ban**: Automated brute-force protection
7. **Tunneling**: Local/remote port forwarding

## Attack Vectors Addressed

| Attack | Mitigation |
|---|---|
| Brute force | Keys + Fail2Ban |
| Root compromise | `PermitRootLogin no` |
| Password spray | `PasswordAuthentication no` |
| Port scanning | Non-default port |
| MITM | Host key verification |
| Idle hijack | ClientAlive timeouts |

# 🎯 Learning Objectives: Did We Achieve?

By now, you should be able to:

- ✅ Explain how SSH works (key exchange, encryption, authentication)
- ✅ Generate Ed25519 key pairs and deploy public keys
- ✅ Configure `~/.ssh/config` for connection aliases
- ✅ Harden `sshd_config` with security best practices
- ✅ Change the default port and restrict user access
- ✅ Set up Fail2Ban for brute-force protection
- ✅ Use SSH tunneling for secure port forwarding
- ✅ Test configuration changes without locking yourself out

🎓 **Next Week:** Firewalls & Packet Filtering – iptables, nftables, firewalld, and UFW!

# 🧪 Lab Practice: SSH Hardening

## Exercise 1: Key Generation & Deployment

Generate an Ed25519 key pair with a passphrase. Deploy it to your lab VM using `ssh-copy-id`. Verify key-based login works.

## Exercise 2: Harden sshd_config

Apply these settings: disable root login, disable passwords, change port to 2222, set `MaxAuthTries 3`, add `AllowUsers` for your user. Test with a second session before closing the first.

## Exercise 3: Fail2Ban Setup

Install Fail2Ban, configure the SSH jail with `maxretry=3` and `bantime=600`. Trigger a ban with intentional failed logins from another terminal. Verify with `fail2ban-client status sshd`.

## Exercise 4: SSH Config & Tunneling

Create `~/.ssh/config` entries for your lab VMs. Set up a local port forward to access a remote web service through SSH. Verify encrypted tunnel with `ss -tlnp`.

# 🔗 Additional Resources

## Documentation

- OpenSSH Manual Pages – Official sshd_config reference
- ssh-audit on GitHub – SSH server & client auditing
- Fail2Ban Documentation – Brute-force protection

## Books & Guides

- *"SSH Mastery"* by Michael W. Lucas – Comprehensive SSH reference
- Mozilla SSH Guidelines – Production hardening
- CIS Benchmark for Linux – SSH section

## Practice

- OverTheWire: Bandit – SSH-based wargame
- Set up two VMs: harden one, attack the other with `hydra`

# Questions?

Next: Firewalls & Packet Filtering